Lab 05: Web Architecture and Protocols

INFO 4617/5617 · Week 5 · Chapter 5, Web Architecture and Protocols

This week every class is live coding: one lab in three parts, one a day. Each step says what we're building and why. Its code cells hold a comment that says what to write, and we write it together. Cells that start `# Given:` are already written: we read them together and run them. Nothing here is turned in.

We follow one page, the chapter's University of Colorado Boulder article, from the browser down to the network and back:

- **Part 1, Monday:** the page in your browser's developer tools, the one request `requests` makes for it, and the name that request gives the server;
- **Part 2, Wednesday:** how a name becomes an address, what happens before the first byte of HTTP, and how to tell from a response what answered;
- **Part 3, Friday:** a URL taken apart, the pageviews API's URL built from its parts, and a month of the article's readers as a plot.

Short ⏱️ microsprints ask you to change one thing, alone or in pairs, and share what you find. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

Before class: open this notebook in the `webdata` environment (chapter 1; it includes `dnspython`), and have your email address ready for the User-Agent. Open Chrome, Firefox, or Safari too: Steps 1, 2, 3 and 7 use its developer tools. In Safari, first turn on **Settings → Advanced → Show features for web developers**. No keys or accounts are needed.

On Wednesday and Friday, start at that day's setup step.

> **Instructor key.** Rebuilt from scratch on 2026-11-11 for the next offering, from chapter 5 in the textbook at commit `3036573`, after the audit in issue 179. Run on 2026-11-11 from 07:04 to 07:10 UTC, in Python 3.14 with requests 2.34.2 and dnspython 2.9.2, from the course's cloud machine, which reaches the web through a proxy. Wikimedia answered some requests with 429, and `labs/tools/run_key.py` reruns only a cell that raises an error; in this lab every request prints its status, so a 429 doesn't raise. The key was therefore run by a copy of `run_key.py`'s procedure that also reran, after a minute, any cell whose own output showed a 429 (five minutes of waiting in this run). Every cell ran and none is tagged `not-run-live`. The saved copies in `labs/week-05/data/` were written by this run, by two run-only cells that have since been removed. What a laptop on campus shows instead: Step 6's DNS answer and Step 8's `x-cache` describe the cloud machine's resolver and proxy, which were sent to codfw (Carrollton, Texas); a laptop in Boulder may be sent to eqiad (208.80.154.224, `cp1…`), and its resolver's TTL counts down from 180 where this one's stayed at 180. Step 2's microsprint counts (47 and 984) and Step 3's browser User-Agent come from headless Chrome on the cloud machine, not a person's browser, and Step 7's two cells are empty because the cloud machine has no browser window: fill them in from a laptop in class. A rerun should match the pageviews (657 on January 1, 902 at the peak on January 22, 20,822 in the month), the 403, 404 and 301, and the `AC/DC` pair (404 ”invalid route”, then 200). It won't necessarily match the `Server` and `x-cache` names, the pages' sizes to the byte, or the version of `requests` in the User-Agent.

Part 1 — Monday: one page, and the one request in it

Your browser makes dozens of requests to show one page; `requests` makes one. Today we find that one in the browser's developer tools, make it in Python, read what came back, and see what name it gives the server, and what happens when it gives the wrong one.

Step 0 — Set up (4 minutes)

Put your email address in `CLIENT`, and run the given cell. It builds the User-Agent that Wikimedia's User-Agent policy asks for: a name and version, a way to reach you, then the library and its version. `HEADERS` is a **dictionary**, pairs of a key and a value inside `{}`, and every request today sends it. An **f-string**, a string with `f` before its quote, fills each `{name}` in it with that name's value.

```
# Given: put your email address in CLIENT, then run this cell
import requests
from bs4 import BeautifulSoup

CLIENT = "WebDataScience/1.0 (your-email@colorado.edu)"
HEADERS = {"User-Agent": f"{CLIENT} python-requests/{requests.__version__}"}
ARTICLE = "University_of_Colorado_Boulder"
PAGE = f"https://en.wikipedia.org/wiki/{ARTICLE}"
print(HEADERS)
print(PAGE)
```

```
{'User-Agent': 'WebDataScience/1.0 (your-email@colorado.edu) python-requests/2.34.2'}
https://en.wikipedia.org/wiki/University_of_Colorado_Boulder
```

💬 Discuss

- In week 2 we wrote a User-Agent with an email address in it. Who reads it, and what could they do with it?
- Ours ends with `python-requests/` and its version. Why would a site want to know the library, as well as who you are?

Step 1 — The title, in the browser and in Python (10 minutes)

Open the article, right-click its title, and choose **Inspect** (**Inspect Element** in Safari). The Inspector (**Elements** in Chrome and Safari) shows the **DOM**, the browser's live tree of the page, at the title's **node**. Right-click the node: **Copy → Copy selector** gives `#firstHeading`, a **selector**, a pattern that picks out the element whose `id` is `firstHeading`. Then request the page in Python, print what came back before reading it, and find the title with `select_one()`. See "The Inspector Tab".

```
# Request PAGE with headers=HEADERS, as article_response
# Print its status code and its first 200 characters, article_response.text[:200]
article_response = requests.get(PAGE, headers=HEADERS)
print(article_response.status_code, article_response.text[:200])
```

```
200 <!DOCTYPE html>
<html class="client-nojs vector-feature-language-in-header-enabled
 ↪  vector-feature-language-in-main-menu-disabled
 ↪  vector-feature-language-in-main-page-header-disabled vector-feature-pag
```

```
# soup: BeautifulSoup(article_response.text, "lxml")
# Print the text of the element soup.select_one("#firstHeading") finds: .text
soup = BeautifulSoup(article_response.text, "lxml")
print(soup.select_one("#firstHeading").text)
```

```
University of Colorado Boulder
```

💬 Discuss

- The Inspector shows the DOM; `requests` gets what View Source shows, as week 1's side-by-side figure did. If the title had been missing from `article_response.text`, what would that tell you?
- Why is an `id` like `firstHeading` a safer target for a scraper than a class name chosen for styling?
- In week 4 we found tags with `find_all()`. When is a copied selector quicker, and when is it more brittle?

Step 2 — The Network tab: one request among many (14 minutes)

Open the Network tab and reload: it records only while open. The status bar counts the requests; the first row is the HTML document. Click it and read its **Headers**, labeled lines sent before the content, both ways. Python reads them by name; `.get()` gives `None` for a missing one. Never print them all: one, `x-client-ip`, holds your public address. `Content-Length` is the size sent, compressed; `len(article_response.content)`, the size unpacked. See "The Network Tab".

```
# Print how many headers came back (not the headers themselves): len(article_response.headers)
print(len(article_response.headers))
```

```
25
```

```
# Print two headers by name: article_response.headers["Content-Type"], then ["Server"]
print(article_response.headers["Content-Type"])
print(article_response.headers["Server"])
```

```
text/html; charset=UTF-8
mw-web.codfw.main-6dc4bb745-bqv6x
```

```
# Print the size the server sent, article_response.headers.get("Content-Length"),
# then the size after requests unpacked it, len(article_response.content)
print(article_response.headers.get("Content-Length"))
print(len(article_response.content))
```

```
121695
767440
```

```
# Print the headers requests sent with it: article_response.request.headers
print(article_response.request.headers)
```

```
{'User-Agent': 'WebDataScience/1.0 (your-email@colorado.edu) python-requests/2.34.2',
 ↪  'Accept-Encoding': 'gzip, deflate, zstd', 'Accept': '*/*', 'Connection': 'keep-alive'}
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: a short version of Additional Exercise 1. Open a news site you read, open the Network tab, reload, and read the number of requests in the status bar. Do the same for the article. (An ad blocker lowers the count; say if you use one.) **Share:** the site, its number of requests, and how many times the article's it is.

```
# Microsprint: type your counts as article_requests and news_requests; print news_requests /
 ↪  article_requests
article_requests = 47  # the key's: one load in headless Chrome, 2026-11-11
news_requests = 984  # the key's: the Daily Camera's home page, the same way
print(news_requests / article_requests)
```

```
20.93617021276596
```

💬 Discuss

- Your browser made dozens of requests for the article; `requests` made one. What does a scraper never see, and what does it spare the server?
- `Content-Length` was about a sixth of `len(article_response.content)` in the key's run. Which one is closer to the Network tab's **Size** column, and why does the server shrink the page?
- Wikipedia sends your own public address back to you in `x-client-ip`. Why is that a reason never to print a response's headers whole, in a notebook you might share?

Step 3 — What the server received (10 minutes)

`article_response.request.headers` is what `requests` sent. httpbin.org shows what a server received: it **echoes**, sending back what you sent. Its `/headers` address answers with your request's headers, and `/user-agent` with the User-Agent alone. Ask the first with `HEADERS`, and the second with no headers at all, so that `requests` uses its own default name. Ask only these two: `/get` and `/ip` echo your public address too. See the chapter's "User-Agent Spoofing".

```
# Send "https://httpbin.org/headers" with headers=HEADERS, as echo_response
# Print its status code and its text
echo_response = requests.get("https://httpbin.org/headers", headers=HEADERS)
print(echo_response.status_code, echo_response.text)
```

```
200 {
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate, zstd",
    "Host": "httpbin.org",
    "User-Agent": "WebDataScience/1.0 (your-email@colorado.edu) python-requests/2.34.2"
  }
}
```

```
# Send "https://httpbin.org/user-agent" with no headers, as default_echo; print its text
default_echo = requests.get("https://httpbin.org/user-agent")
print(default_echo.text)
```

```
{
  "user-agent": "python-requests/2.34.2"
}
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: open https://httpbin.org/headers in your browser: it shows what your browser sent. Count its headers, and copy its `User-Agent` into the cell, as a comment. **Share:** how many headers your browser sent, against `requests`' four, and which browsers its User-Agent names.

```
# Microsprint: your browser's number of headers, and its User-Agent, as comments
# The key's: headless Chrome on the course's cloud machine sent 14 headers, and this User-Agent:
# Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/153.0.0.0
↪  Safari/537.36
```

💬 Discuss

- `requests` sent four headers; your browser sent many more. Which of the browser's could a site use to tell a person's browser from a program?
- Chrome's User-Agent starts with `Mozilla` and ends with `Safari`. The chapter's "User-Agent Spoofing" says why. What does that tell you about trusting this header?
- httpbin answers plain `http://` too, with no error and no redirect. Who could read your User-Agent, email and all, on the way?

Step 4 — The default name, refused, and the rule (12 minutes)

Now send the article's request with no headers, so it carries `requests`' default name. Wikimedia's policy: User-Agents "that begin with non-descriptive default values, such as python-requests/x, may also be blocked". A 403 means refused. The given cell sends a **spoofed** User-Agent, one claiming to be a browser, to httpbin only. The policy is plain: "Do not copy a browser's user agent for your bot." In this course we send our own name; if a site refuses it, we stop and ask.

```
# Request PAGE with no headers, as anonymous_response
# Print its status code and its first 200 characters
anonymous_response = requests.get(PAGE)
print(anonymous_response.status_code, anonymous_response.text[:200])
```

```
403 Please set a user-agent and respect our robot policy https://w.wiki/4wJS. See also
 ↪  https://phabricator.wikimedia.org/T400119.
```

```
# Print who answered: anonymous_response.headers["Server"]
print(anonymous_response.headers["Server"])
```

```
HAProxy
```

```
# Given: the chapter's spoofed User-Agent, which claims to be Chrome on Windows, sent only to
 ↪  httpbin's echo,
# never to Wikimedia. Two strings side by side inside ( ) are joined into one
chrome_claim = {"User-Agent": ("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
                               "(KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36")}
claim_echo = requests.get("https://httpbin.org/user-agent", headers=chrome_claim)
print(claim_echo.text)
```

```
{
  "user-agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
   ↪  Chrome/128.0.0.0 Safari/537.36"
}
```

💬 Discuss

- Who refused us, and how do you know? In week 3 we read the `server` header and the body before blaming a platform.
- Our own User-Agent ends with the same `python-requests/` and version, and Wikimedia answered it. Why, by the policy's wording?
- Wikimedia's policy says that bot-like behavior with a browser's User-Agent "will be assumed malicious". Why would a site treat a copied browser string as worse than no name at all?

Part 2 — Wednesday: a name, an address, and what answered

Before a browser or `requests` sends a byte of HTTP, three things happen. DNS turns the name into an **IP address**, the number that identifies a machine on the internet; TCP opens a connection to that address; and TLS encrypts the connection. Then the request goes, and a response comes back with a status code. Today we watch each stage, and learn to tell from a response what answered.

Step 5 — Set up again (3 minutes)

Part 2 starts fresh, so a student who missed Monday can start here. Put your email address in `CLIENT`, and run the given cell. It also imports dnspython's resolver, which asks DNS our questions, and `time`, for a pause between requests.

```
# Given: Part 2's setup. Put your email address in CLIENT, then run it
import time
import requests
import dns.resolver
from bs4 import BeautifulSoup

CLIENT = "WebDataScience/1.0 (your-email@colorado.edu)"
```

```
HEADERS = {"User-Agent": f"{CLIENT} python-requests/{requests.__version__}"}
ARTICLE = "University_of_Colorado_Boulder"
PAGE = f"https://en.wikipedia.org/wiki/{ARTICLE}"
print(HEADERS)
```

```
{'User-Agent': 'WebDataScience/1.0 (your-email@colorado.edu) python-requests/2.34.2'}
```

💬 **Discuss**

- On Monday the 403 came back as a sentence of plain text. If our code had read it with `.json()`, as if it were data, what would we have seen instead of the server's words?

Step 6 — DNS: a name becomes an address (12 minutes)

The **Domain Name System** (DNS) is the internet's address book. A **resolver**, the DNS server your computer asks, answers with records. `dns.resolver.resolve()` asks for one type: `"A"` records hold IPv4 addresses, four numbers with dots. The given cell prints each address with a **`for` loop**, which runs its indented lines once for each item; then the name the record belongs to, and its **TTL**, the seconds a resolver may keep the answer. See *DNS: The Address Book*.

```
# answers: dns.resolver.resolve("en.wikipedia.org", "A")
# Print the first address in the answer, answers[0]
answers = dns.resolver.resolve("en.wikipedia.org", "A")
print(answers[0])
```

```
208.80.153.224
```

```
# Given: every address in the answer (the chapter's loop), the name the record belongs to, and
 ↪  its TTL
for answer in answers:
    print(f"IP address: {answer}")
print("Canonical name:", answers.canonical_name)
print("TTL:", answers.rrset.ttl, "seconds")
```

```
IP address: 208.80.153.224
Canonical name: dyna.wikimedia.org.
TTL: 180 seconds
```

```
# On purpose, a mistake: give resolve() the whole address, PAGE, with "A"
# Read the error's last line: what does DNS say about it?
dns.resolver.resolve(PAGE, "A")
```

```
---------------------------------------------------------------------------
NXDOMAIN                                  Traceback (most recent call last)
Cell In[18], line 3
      1 # On purpose, a mistake: give resolve() the whole address, PAGE, with "A"
      2 # Read the error's last line: what does DNS say about it?
----> 3 dns.resolver.resolve(PAGE, "A")

File /opt/webdata/lib/python3.14/site-packages/dns/resolver.py:1557, in resolve(qname, rdtype,
 ↪  rdclass, tcp, source, raise_on_no_answer, source_port, lifetime, search)
   1537 def resolve(
   1538     qname: dns.name.Name | str,
   1539     rdtype: dns.rdatatype.RdataType | str = dns.rdatatype.A,
   (...)   1546     search: bool | None = None,
   1547 ) -> Answer:  # pragma: no cover
   1548     """Query nameservers to find the answer to the question.
   1549
```

```
   1550     This is a convenience function that uses the default resolver
   (...)   1554     parameters.
   1555     """
-> 1557     return get_default_resolver().resolve(
   1558         qname,
   1559         rdtype,
   1560         rdclass,
   1561         tcp,
   1562         source,
   1563         raise_on_no_answer,
   1564         source_port,
   1565         lifetime,
   1566         search,
   1567     )

File /opt/webdata/lib/python3.14/site-packages/dns/resolver.py:1303, in Resolver.resolve(self,
 ↪  qname, rdtype, rdclass, tcp, source, raise_on_no_answer, source_port, lifetime, search)
   1301 start = time.time()
   1302 while True:
-> 1303     request, answer = resolution.next_request()
   1304     # Note we need to say "if answer is not None" and not just
   1305     # "if answer" because answer implements __len__, and python
   1306     # will call that.  We want to return if we have an answer
   1307     # object, including in cases where its length is 0.
   1308     if answer is not None:
   1309         # cache hit!

File /opt/webdata/lib/python3.14/site-packages/dns/resolver.py:759, in
 ↪  _Resolution.next_request(self)
    752     return (request, None)
    754 #
    755 # We've tried everything and only gotten NXDOMAINs.  (We know
    756 # it's only NXDOMAINs as anything else would have returned
    757 # before now.)
    758 #
--> 759 raise NXDOMAIN(qnames=self.qnames_to_try, responses=self.nxdomain_responses)

NXDOMAIN: The DNS query name does not exist:
 ↪  https://en.wikipedia.org/wiki/University_of_Colorado_Boulder.
```

💬 Discuss

- DNS said the name `https://en.wikipedia.org/wiki/...` doesn't exist. What is the difference between a URL and a name, and which part of the URL is the name?
- A resolver may keep this answer for the TTL's seconds. If Wikipedia moved to a new address tomorrow, how long could some of us still be sent to the old one?
- Try the name misspelled, `en.wikipeda.org`. Does DNS warn you? If not, how would you notice?

Step 7 — Below HTTP: one request, in stages (7 minutes)

Under HTTP, **TCP** opens a connection between two addresses with three small packets, the handshake; for `https`, **TLS** then sets up encryption. Only then does the request go. Your browser times each stage: in the Network tab, click the article's first row, then **Timing**. Find the DNS lookup, the connection, the TLS (SSL) setup, and the wait for the server's first byte; each browser names them a little differently. See the chapter's "Packets".

```
# Write the Timing tab's times here, as comments: DNS lookup, connection, SSL (TLS), waiting
# (The key has none: the course's cloud machine has no browser window, and its requests go
 ↪  through a
# proxy. The instructor's laptop fills these in, in class.)
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: reload the page, click the first row again, and read its **Timing** a second time. **Share:** which stages had a time on the first load and none (or almost none) on the second, and a guess at why.

```
# Microsprint: the second load's times, as comments
# (From a laptop in class, as above)
```

💬 Discuss

- If the DNS lookup took no time the second time, where did the address come from? Step 6's TTL is a hint.
- A browser keeps a connection open for the next request to the same host. What would that save a scraper that sends a hundred requests? The chapter's "Connections, Sessions, and Retries" shows how Python does it.
- TCP's three packets come before any HTTP. If no server answered them, what would `requests` give you instead of a status code?

Step 8 — What answered: status, Server, and the cache (12 minutes)

A status code is the server's answer about a request: 200, it worked; 404, nothing at this address; 403, refused; 429, too many requests, slow down. Ask for a page that doesn't exist. Then read two headers: `Server` names the program that built the page, and `x-cache` names Wikimedia's **cache**, the server that keeps copies of pages so it can answer without building them again. See "Diagnosing HTTP Errors".

```
# missing_url: the chapter's missing page, "https://en.wikipedia.org/wiki/Nonexistent_Page_12345"
missing_url = "https://en.wikipedia.org/wiki/Nonexistent_Page_12345"
```

```
# Request missing_url with HEADERS, as missing_response
# Print its status code and its "Content-Type" header
missing_response = requests.get(missing_url, headers=HEADERS)
print(missing_response.status_code, missing_response.headers["Content-Type"])
```

```
404 text/html; charset=UTF-8
```

```
# Print the size of what came back: len(missing_response.content)
print(len(missing_response.content))
```

```
53030
```

```
# Print two headers: missing_response.headers["Server"], then ["x-cache"]
print(missing_response.headers["Server"])
print(missing_response.headers["x-cache"])
```

```
mw-web.codfw.main-6dc4bb745-l2krv
cp2045 miss, cp2045 hit/1
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: which of Wikimedia's data centers answered you? The given cell asks DNS for each one's address. Find Step 6's address among them. Then read the number in `x-cache` (`cp2045` is 2045):

Wikimedia numbers its servers by data center, 1000–1999 in eqiad, 2000–2999 in codfw, and so on. **Share:** your data center by its address, and by `x-cache`. Do the two agree?

```
# Given: each data center's front door, text-lb.<site>.wikimedia.org: its address, and its
 ↪  servers' numbers
sites = {"eqiad": "1000s, Ashburn, Virginia", "codfw": "2000s, Carrollton, Texas",
         "esams": "3000s, Amsterdam", "ulsfo": "4000s, San Francisco", "eqsin": "5000s,
          ↪  Singapore",
         "drmrs": "6000s, Marseille", "magru": "7000s, São Paulo"}
for site in sites:
    site_answer = dns.resolver.resolve(f"text-lb.{site}.wikimedia.org", "A")
    print(site, site_answer[0], sites[site])
```

```
eqiad 208.80.154.224 1000s, Ashburn, Virginia
codfw 208.80.153.224 2000s, Carrollton, Texas
esams 185.15.59.224 3000s, Amsterdam
ulsfo 198.35.26.224 4000s, San Francisco
eqsin 103.102.166.224 5000s, Singapore
drmrs 185.15.58.224 6000s, Marseille
magru 195.200.68.224 7000s, São Paulo
```

```
# Microsprint: print Step 6's address, answers[0], and missing_response.headers["x-cache"] again
print(answers[0])
print(missing_response.headers["x-cache"])
```

```
208.80.153.224
cp2045 miss, cp2045 hit/1
```

💬 Discuss

- The 404 came back as a whole page, with a size and a Content-Type. Why can't a scraper take "some HTML came back" to mean success?
- `Server` and `x-cache` name two different machines. Which one did DNS send us to, and which one built the page? In week 3, which header told us who had refused us?
- In the key's run, the address and `x-cache` named the same data center. Would a classmate on a phone hotspot get the same one, and does it change the data we collect?

Step 9 — Redirects, two kinds (8 minutes)

A **redirect** sends you elsewhere. A 301 says a page has moved for good, and its `Location` header says where; `requests` follows it, and keeps each step in `response.history`. Ask for `CU_Boulder` over plain `http://`. The server moves you to `https://`. Then Wikipedia, which keeps `CU_Boulder` as another name for the article, answers with the article itself. The given cell stops at the first step, with `allow_redirects=False`.

```
# alias_url: "http://en.wikipedia.org/wiki/CU_Boulder" (http, not https)
alias_url = "http://en.wikipedia.org/wiki/CU_Boulder"
```

```
# Request alias_url with HEADERS, as alias_response; print its status code and alias_response.url
alias_response = requests.get(alias_url, headers=HEADERS)
print(alias_response.status_code, alias_response.url)
```

```
200 https://en.wikipedia.org/wiki/CU_Boulder
```

```
# Print the redirects requests followed on the way: alias_response.history
print(alias_response.history)
```

```
[<Response [301]>]
```

```
# alias_soup: BeautifulSoup(alias_response.text, "lxml"); print the text of its "#firstHeading"
alias_soup = BeautifulSoup(alias_response.text, "lxml")
print(alias_soup.select_one("#firstHeading").text)
```

```
University of Colorado Boulder
```

```
# Given: the first step only. allow_redirects=False stops requests from following the 301
first_hop = requests.get(alias_url, headers=HEADERS, allow_redirects=False)
print(first_hop.status_code, first_hop.headers["Location"], first_hop.headers["Server"])
```

```
301 https://en.wikipedia.org/wiki/CU_Boulder HAProxy
```

💬 Discuss

- `alias_response` said 200, and its history held a 301. What would a check of the status code alone have missed?
- We asked for `CU_Boulder` and got a page headed with another title. How could a scraper that files pages under the title it asked for go wrong?
- Why do sites move `http://` visitors to `https://`? Step 7's stages are a hint. (If you typed `https://`, your history was empty: nothing needed moving.)

Step 10 — The chapter's `diagnose_request()`, read together (8 minutes)

A **function** is lines with a name, which run each time it's called with `( )`. `def` defines one; its **parameters**, here `url` and `headers`, are filled in by each call. The chapter's `diagnose_request()` gathers what Steps 8 and 9 printed. It's given without the chapter's `try`/`except` and `timeout=`, which keep a dead address from stopping the notebook. Find the lines you typed in it, then run it on our three addresses. See "Diagnosing HTTP Errors".

```
# Given: the chapter's diagnose_request(), without its try/except and timeout
def diagnose_request(url, headers=None):
    """Print diagnostic information about an HTTP request."""
    response = requests.get(url, headers=headers)
    print(f"URL: {url}")
    print(f"Status: {response.status_code}")
    print(f"Content-Type: {response.headers.get('Content-Type', 'not specified')}")
    print(f"Server: {response.headers.get('Server', 'not specified')}")
    print(f"Response size: {len(response.content):,} bytes")
    if response.history:
        print(f"Redirected from: {response.history[0].url}")
    return response
```

```
# Given: the function on the three addresses we requested by hand, a second apart
for url in [PAGE, missing_url, alias_url]:
    diagnose_request(url, headers=HEADERS)
    print()
    time.sleep(1)
```

```
URL: https://en.wikipedia.org/wiki/University_of_Colorado_Boulder
Status: 200
Content-Type: text/html; charset=UTF-8
Server: mw-web.codfw.main-6dc4bb745-bqv6x
Response size: 767,440 bytes


URL: https://en.wikipedia.org/wiki/Nonexistent_Page_12345
Status: 404
Content-Type: text/html; charset=UTF-8
```

```
Server: mw-web.codfw.main-6dc4bb745-wstg6
Response size: 53,030 bytes


URL: http://en.wikipedia.org/wiki/CU_Boulder
Status: 200
Content-Type: text/html; charset=UTF-8
Server: mw-web.codfw.main-6dc4bb745-8trhj
Response size: 768,052 bytes
Redirected from: http://en.wikipedia.org/wiki/CU_Boulder
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: httpbin answers `https://httpbin.org/status/` followed by a number with that status. Choose one from MDN's list (418 is a joke), and run `diagnose_request()` on it, with `HEADERS`. **Share:** your status, its Content-Type, and its size. Which statuses came back with nothing in them?

```
# Microsprint: diagnose_request("https://httpbin.org/status/418", headers=HEADERS), with your
↪  number
diagnose_request("https://httpbin.org/status/418", headers=HEADERS)
```

```
URL: https://httpbin.org/status/418
Status: 418
Content-Type: not specified
Server: not specified
Response size: 135 bytes
```

```
<Response [418]>
```

💬 Discuss

- The chapter's version gives the request `timeout=10` and catches a failed connection. What would ours do with an address that never answers? Recommended Exercise Step 4 asks for the timeout.
- `diagnose_request()` prints where a redirect came from. What else would you want to know about a redirect?
- A 404 can be our mistake, and a 500 is the server's. Whose is a 429, and what should our code do about it?

Part 3 — Friday: a URL in parts, and a month of readers

A URL isn't just text: it has parts, and each one means something to the server. Today we take one apart with Python's `urllib.parse`, build the pageviews API's URL from its parts, and turn its answer into a plot of the article's readers in January 2026, the chapter's month.

Step 11 — Set up again (3 minutes)

Part 3 starts fresh. Put your email address in `CLIENT`, and run the given cell. It also sets `PAGEVIEWS`, the pageviews API's address up to the article, and `saved`, the address of the course's saved copies of today's answers, for a student the API refuses.

```
# Given: Part 3's setup. Put your email address in CLIENT, then run it
import time
import requests
import pandas as pd
import matplotlib.pyplot as plt
from urllib.parse import urlparse, parse_qs, quote
```

```
CLIENT = "WebDataScience/1.0 (your-email@colorado.edu)"
HEADERS = {"User-Agent": f"{CLIENT} python-requests/{requests.__version__}"}
ARTICLE = "University_of_Colorado_Boulder"
PAGEVIEWS =
 ↪  "https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/en.wikipedia/all-access/all-agents"
saved =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-05/data/"
print(HEADERS)
```

```
{'User-Agent': 'WebDataScience/1.0 (your-email@colorado.edu) python-requests/2.34.2'}
```

💬 Discuss

- `PAGEVIEWS` is on `wikimedia.org`, not `wikipedia.org`. How would you check, with Wednesday's code, whether the two names lead to the same address?

Step 12 — Take a URL apart (8 minutes)

A URL's parts are `scheme://host/path?query#fragment`. The **scheme** says how to ask (`https`); the **host**, which Python calls `netloc`, is the name DNS resolves; the **path** says what, on that host; the **query**, after `?`, holds `key=value` pairs. Don't split URLs by hand: `urlparse()` knows their grammar, and `parse_qs()` turns a query into a dictionary. See *URLs: Anatomy and Parsing*.

```
# Given: the chapter's address for a Census Bureau API: Boulder's name and population, from
 ↪  2022's survey
census_url =
 ↪  "https://api.census.gov/data/2022/acs/acs5?get=NAME,B01001_001E&for=place:07850&in=state:08"
```

```
# parsed: urlparse(census_url); print parsed.scheme, then parsed.netloc
parsed = urlparse(census_url)
print(parsed.scheme)
print(parsed.netloc)
```

```
https
api.census.gov
```

```
# Print parsed.path, then parsed.query
print(parsed.path)
print(parsed.query)
```

```
/data/2022/acs/acs5
get=NAME,B01001_001E&for=place:07850&in=state:08
```

```
# Turn the query into a dictionary, and print it: parse_qs(parsed.query)
print(parse_qs(parsed.query))
```

```
{'get': ['NAME,B01001_001E'], 'for': ['place:07850'], 'in': ['state:08']}
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: take apart an address copied without its `https://`, as people often copy one: `"en.wikipedia.org/wiki/University`
Print the whole result. **Share:** what `netloc` holds, and where the host went.

```
# Microsprint: no_scheme: urlparse("en.wikipedia.org/wiki/University_of_Colorado_Boulder"); print
 ↪  it
no_scheme = urlparse("en.wikipedia.org/wiki/University_of_Colorado_Boulder")
print(no_scheme)
```

```
ParseResult(scheme='', netloc='', path='en.wikipedia.org/wiki/University_of_Colorado_Boulder',
 ↪  params='', query='', fragment='')
```

💬 Discuss

- `parse_qs()` gave each value as a list, such as `['place:07850']`. Why might a query hold the same key twice?
- `urlparse()` didn't complain about the address without `https://`: it put the host in the path. Where would that silent mistake show up later, if you passed its `netloc` to Wednesday's `dns.resolver.resolve()`?
- Try giving `parse_qs()` the whole address, `census_url`, instead of `parsed.query`. Does it complain? What is its first key, and why can't it tell?

Step 13 — Build the pageviews URL, and ask (10 minutes)

The pageviews API puts every value in the path, between slashes, in a fixed order: project, access and agent (in `PAGEVIEWS`), then the article, `daily`, and the first and last days, as `YYYYMMDD`. Build the URL with an f-string, send it, and look before reading. `.json()` turns the answer's text into dictionaries and lists. The given cell shows why a title with a slash needs `quote()`. See "Building URLs Programmatically".

```
# api_url: f"{PAGEVIEWS}/{ARTICLE}/daily/20260101/20260131"; print it
api_url = f"{PAGEVIEWS}/{ARTICLE}/daily/20260101/20260131"
print(api_url)
```

```
https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/en.wikipedia/all-access/all-agen
 ↪  ts/University_of_Colorado_Boulder/daily/20260101/20260131
```

```
# Request api_url with HEADERS, as api_response
# Print its status code and its first 200 characters
api_response = requests.get(api_url, headers=HEADERS)
print(api_response.status_code, api_response.text[:200])
```

```
200 {"items":[{"project":"en.wikipedia","article":"University_of_Colorado_Boulder","granularity"
 ↪  :"daily","timestamp":"2026010100","access":"all-access","agent":"all-agents","views":657},{"
 ↪  project":"en.wik
```

```
# data: api_response.json(); print its keys()
data = api_response.json()
print(data.keys())
```

```
dict_keys(['items'])
```

```
# Print the first day: data["items"][0]
print(data["items"][0])
```

```
{'project': 'en.wikipedia', 'article': 'University_of_Colorado_Boulder', 'granularity': 'daily',
 ↪  'timestamp': '2026010100', 'access': 'all-access', 'agent': 'all-agents', 'views': 657}
```

```
# Given, only if a cell above stopped: remove the "#" from the next line. It loads the same
 ↪  answer, saved 2026-11-11
# data = requests.get(saved + "pageviews-cu-boulder-2026-01.json", headers=HEADERS).json()
```

```
# Given: the chapter's warning about slashes. The same month for "AC/DC": the title raw, then
 ↪  with
# quote(title, safe=""), which writes the slash as %2F. A second apart; print the status and the
 ↪  body's start
for title in ["AC/DC", quote("AC/DC", safe="")]:
    acdc_url = f"{PAGEVIEWS}/{title}/daily/20260101/20260131"
    acdc_response = requests.get(acdc_url, headers=HEADERS)
```

```
    print(title, acdc_response.status_code, acdc_response.text[:100])
    time.sleep(1)
```

```
AC/DC 404 {"detail":"invalid route","method":"get","status":404,"title":"Not
↪  Found","type":"about:blank","uri"

AC%2FDC 200 {"items":[{"project":"en.wikipedia","article":"AC/DC","granularity":"daily","timesta
↪  mp":"2026010100"
```

💬 Discuss

- Why put a fixed range, January 2026, in the URL, rather than ask for "the last 30 days"?
- What did the server make of the raw slash in `AC/DC`, and which words in its answer say so?
- If the API had refused you, the saved copy would have given you the same numbers. What can't a saved copy tell you that the live request can?

Step 14 — From JSON to a time series (10 minutes)

`data["items"]` is a list of dictionaries, one a day, which `pd.DataFrame()` turns into a table, as in week 4. Its `timestamp` is text: year, month, day and hour (`2026010100`). `pd.to_datetime()` with `format="%Y%m%d00"` makes it a date: `%Y` year, `%m` month (a capital `%M` is minutes), `%d` day. The plot is given. `idxmax()` gives the row label of the largest value, and `.loc[ ]` reads that row.

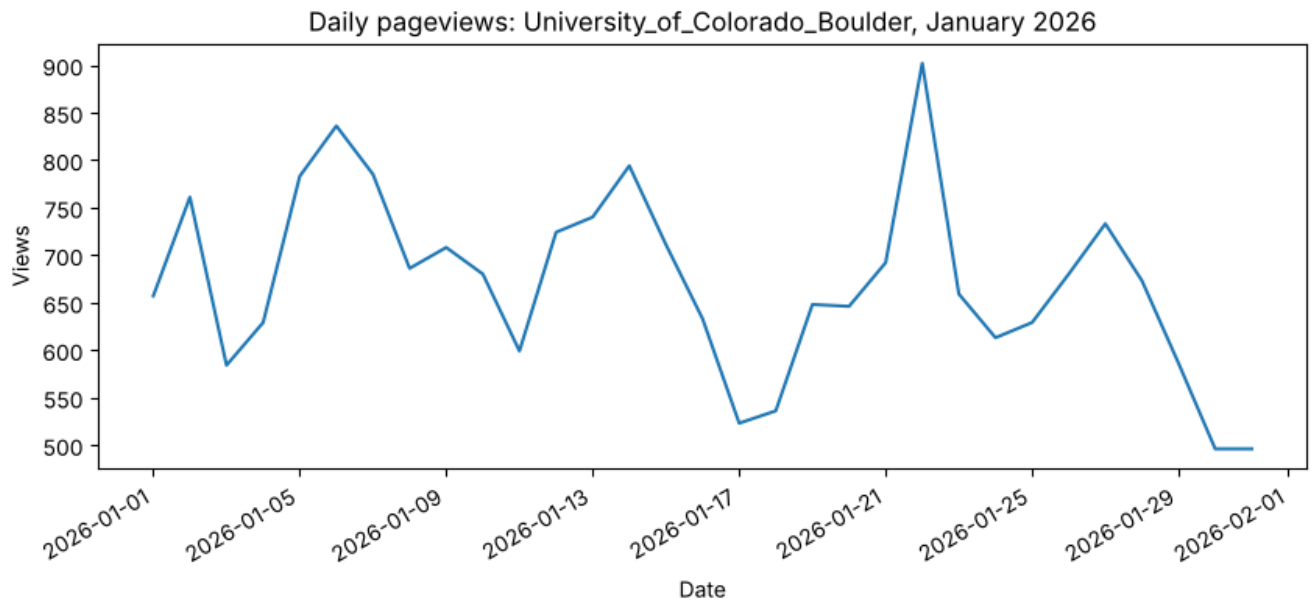
```
# df: pd.DataFrame(data["items"]); print its first rows, df.head()
df = pd.DataFrame(data["items"])
print(df.head())
```

```
        project                          article granularity   timestamp  \
0  en.wikipedia  University_of_Colorado_Boulder       daily  2026010100
1  en.wikipedia  University_of_Colorado_Boulder       daily  2026010200
2  en.wikipedia  University_of_Colorado_Boulder       daily  2026010300
3  en.wikipedia  University_of_Colorado_Boulder       daily  2026010400
4  en.wikipedia  University_of_Colorado_Boulder       daily  2026010500

       access       agent  views
0  all-access  all-agents    657
1  all-access  all-agents    761
2  all-access  all-agents    584
3  all-access  all-agents    629
4  all-access  all-agents    783
```

```
# Turn the timestamps into dates: df["timestamp"] = pd.to_datetime(df["timestamp"],
↪  format="%Y%m%d00")
df["timestamp"] = pd.to_datetime(df["timestamp"], format="%Y%m%d00")
```

```
# Given: views by day, with a title and labeled axes
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(df["timestamp"], df["views"])
ax.set(title=f"Daily pageviews: {ARTICLE}, January 2026", xlabel="Date", ylabel="Views")
fig.autofmt_xdate()
plt.show()
```




```
# Print the row of the day with the most views: df.loc[df["views"].idxmax()]
print(df.loc[df["views"].idxmax()])
```

```
project                          en.wikipedia
article        University_of_Colorado_Boulder
granularity                             daily
timestamp                2026-01-22 00:00:00
access                            all-access
agent                             all-agents
views                                    902
Name: 21, dtype: object
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: a short version of Additional Exercise 3. Put two articles related to ours in `others` (another university, the city, a team), with underscores for spaces, and run the given cells under it. They ask for the same month for all three, a second apart, and plot them on one set of axes. `[ARTICLE] + others` joins two lists into one. **Share:** your two articles, which of the three had the biggest spike, on what day, and a guess at why.

```
# Microsprint: others, a list of two titles, such as ["Colorado_State_University",
↪  "University_of_Denver"]
others = ["Colorado_State_University", "University_of_Denver"]
```

```
# Given: the same month for ARTICLE and the two in others, a second apart, as one table with a
↪  column for each
columns = {}
for title in [ARTICLE] + others:
    title_url = f"{PAGEVIEWS}/{title}/daily/20260101/20260131"
    title_response = requests.get(title_url, headers=HEADERS)
    print(title, title_response.status_code)
    items = pd.DataFrame(title_response.json()["items"])
    columns[title] = items.set_index("timestamp")["views"]
    time.sleep(1)
compare = pd.DataFrame(columns)
compare.index = pd.to_datetime(compare.index, format="%Y%m%d00")
```

```
compare.index.name = "timestamp"
```

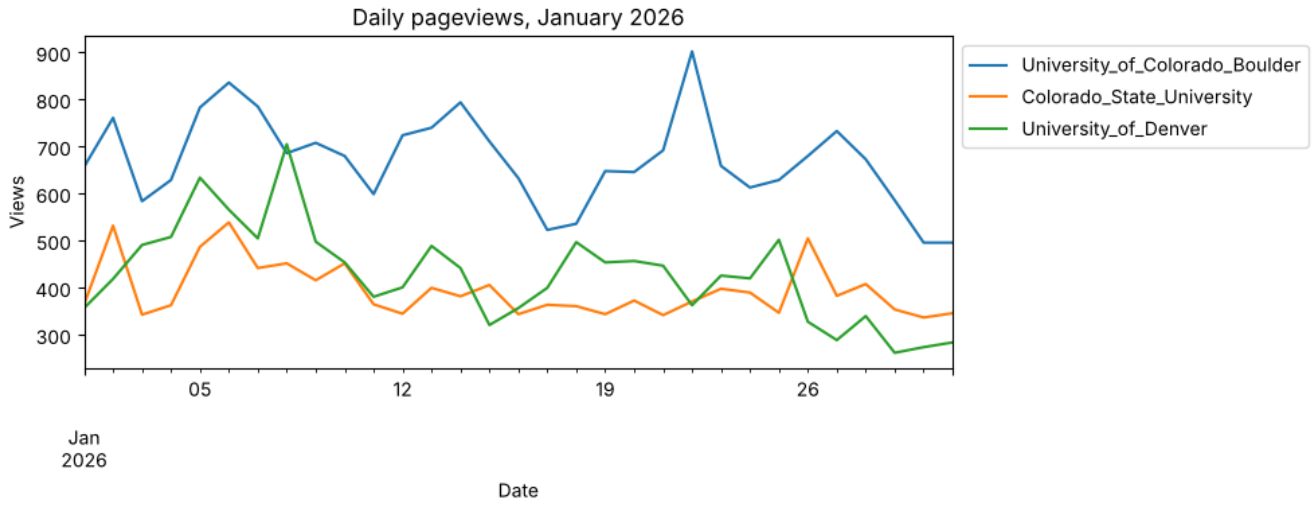
```
University_of_Colorado_Boulder 200
```

```
Colorado_State_University 200
```

```
University_of_Denver 200
```

```
# Given, only if the cell above stopped: remove the "#" from the next two lines. They load the
 ↪  key's three
# articles, saved 2026-11-11
# compare = pd.read_csv(saved + "pageviews-three-articles-2026-01.csv",
#                       index_col="timestamp", parse_dates=["timestamp"])
```

```
# Given: the three on one set of axes, and each one's biggest day
fig, ax = plt.subplots(figsize=(10, 4))
compare.plot(ax=ax)
ax.set(title="Daily pageviews, January 2026", xlabel="Date", ylabel="Views")
ax.legend(loc="upper left", bbox_to_anchor=(1, 1))
fig.tight_layout()
plt.show()
print(compare.idxmax())
```




```
University_of_Colorado_Boulder   2026-01-22
Colorado_State_University        2026-01-06
University_of_Denver             2026-01-08
dtype: datetime64[us]
```

💬 Discuss

- What happened on the article's biggest day? How would you check, rather than guess?
- These are `all-agents` counts, crawlers included. Which claims about attention can the plot support, and which can't it?
- Try one mistake: the title in lower case, `CU_Boulder` (a redirect) as the title, or `%M` for `%m`. Did anything raise an error? Would the plot have told you?

Step 15 — What did we learn? (4 minutes)

In a few sentences, as comments:

- Put these in the order they happen when you load the article: the TLS setup, the HTTP request, the DNS lookup, the page drawn, the TCP connection, the status code. Which of them did we

see from Python, and which only in the browser?
- Which status codes did we meet this week, and which response would a check of the status alone have read wrongly?
- Which of this week's outputs describe your own network or computer, rather than Wikipedia?
- Wikimedia refused one of our requests on Monday. What did it refuse, and what does its policy ask of us instead?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, take a URL apart: Step 12's `urlparse()` and `parse_qs()` of the Census address, and its microsprint's address without a scheme.
- Step 2, resolve a name: Step 6's `dns.resolver.resolve()`, and the given loop that prints each address.
- Step 3, request with named headers: Step 0's `HEADERS`, and Step 2's headers read by name, with `.get("Content-Length")` and `len(response.content)` for when the header is absent.
- Step 4, write `diagnose_url()`: Step 10's `diagnose_request()`, and Step 9's `.history` and `.url`. The exercise adds where a redirect ended up, a docstring of your own, and the `timeout=` we left out.
- Step 5, profile five URLs: Steps 4, 8 and 9 (a refusal, a 404, an `http://` redirect) and Step 13 (an API), with week 3's habit of reading a refusal's body.
- Step 6, interpret: Step 2's Content-Type and sizes, Step 8's `Server` and `x-cache`, and Step 13's JSON beside Step 1's HTML.

```
# Your answers here, as comments
# 1. The DNS lookup, the TCP connection, the TLS setup, the HTTP request, the status code, the
 ↪  page drawn.
#    From Python we saw the DNS answer (Step 6), the request's headers and the statuses (Steps
 ↪  1-4, 8-9);
#    the connection and the TLS setup only in the browser's Timing tab (Step 7), and the drawing
 ↪  only there.
# 2. 200 (the article), 403 (no User-Agent; HAProxy's "Please set a user-agent"), 404 (a
 ↪  53,030-byte page),
#    301 (http to https, inside alias_response.history), 418 (httpbin's joke), and 429 if
 ↪  Wikimedia slowed you down.
#    A status-only check reads CU_Boulder's 200 as the page we asked for, though it came through
 ↪  a 301 and
#    Wikipedia's own redirect, and is headed "University of Colorado Boulder"; "HTML came back"
 ↪  passes the 404.
# 3. The DNS answer (208.80.153.224, codfw, kept 180 s), the cache that answered (cp2045, codfw),
 ↪  the
#    Timing tab's times, and the version of requests in our User-Agent. x-client-ip is our own
 ↪  address,
#    which is why we never printed it.
# 4. The article requested with no headers, so with requests' default name,
 ↪  python-requests/2.34.2: 403.
#    The policy asks for a name, a way to reach us, and the library, and never a copied browser
 ↪  string.
```

Friday: from the lab to the textbook (15 minutes)

Did anything in this lab disagree with chapter 5, or leave you stuck? Places to look:

- **What we left out.** The chapter prints your computer's name and its local and public addresses, and runs `ping`, `traceroute` and `curl` from the notebook. We left them out: their output can identify your computer and your network, and the commands differ by operating system. Should the chapter say what each one reveals before a reader shares a notebook?
- **The chapter's `geolocate_ip()`** asks ip-api over plain `http`, the only kind its free service offers, and with no User-Agent. What should the chapter say about a request that can't be sent privately?

- **Outputs that have moved.** The chapter's Network tab figure counts 37 requests for the article. What did your browser count?
- **A cell that broke for you,** such as `dns.resolver` off campus. Was the chapter's *Common Issues to Debug* enough?
- **A microsprint that would make a good exercise,** such as Step 8's check of which data center answered.

`handouts/common/revision-framework.md` says how to propose a revision.